

Infosys Angular + Angular 20 Interview Q&A;

3 Years Experience • Questions first • Answers separately • Interview in 2–3 hours

Fresh research: Recent Infosys Angular reports mention component communication, JavaScript/arrays, CSS/SCSS, RxJS, Angular versions/core concepts, functional chaining and performance. A recent Infosys Angular report also lists Signals vs Observables, effects, computed signals, signal input/output/ViewChild APIs, change detection, pagination, NgRx and large-grid topics. These are recurring reported themes, not a guarantee of the exact questions tomorrow.

Angular 20: The guide emphasizes standalone components, Signals, input(), HttpClient configuration, control flow, OnPush and modern change-detection concepts from current Angular documentation.

PART 1 — QUESTIONS ONLY

1. What is Angular?
2. What is a component?
3. What are standalone components?
4. NgModule vs standalone?
5. Explain lifecycle hooks.
6. constructor vs ngOnInit?
7. What are Angular binding types?
8. Property vs attribute binding?
9. What are directives?
10. What are pipes? Pure vs impure?
11. Explain Angular DI.
12. What does providedIn:'root' mean?
13. Explain routing.
14. ActivatedRoute snapshot vs paramMap?
15. Path vs query parameter?
16. How do you lazy load in modern Angular?
17. How is HttpClient configured in standalone Angular?
18. What is an HTTP interceptor?
19. How do you attach JWT to every request?
20. What is a Signal in Angular 20?
21. Signal vs normal variable?
22. computed vs effect?
23. What is linkedSignal?
24. Are Signals a replacement for RxJS?
25. When use switchMap?
26. switchMap vs mergeMap vs concatMap vs exhaustMap?
27. What is forkJoin?
28. Subject vs BehaviorSubject?
29. Explain change detection.
30. Default vs OnPush?
31. Signals with OnPush?
32. What is zoneless Angular?
33. Explain @if, @for, @switch.
34. Why track in @for?
35. What is @defer?
36. How optimize a large Angular table?
37. How handle API pagination?
38. What are reactive forms?

- 39. Template-driven vs reactive forms?
- 40. What is ControlValueAccessor?
- 41. What is ViewChild?
- 42. @Input/@Output vs input()/output() in Angular 20?
- 43. Parent-child communication?
- 44. Component vs service responsibility?
- 45. Do you need NgRx for every app?
- 46. How prevent Angular memory leaks?
- 47. What is AOT?
- 48. How debug a slow API/page?
- 49. Explain Angular to Spring Boot flow.
- 50. How explain your Angular 20 project?

PART 2 — MODEL ANSWERS

1. What is Angular?

A TypeScript framework for structured web applications, providing components, DI, routing, forms, HTTP, testing and tooling.

2. What is a component?

The basic UI building block: TypeScript class + template + metadata. Components form a tree.

3. What are standalone components?

Components that declare dependencies directly in imports instead of being declared in an NgModule. In modern Angular they are the default style.

4. NgModule vs standalone?

NgModules group declarations/imports/providers. Standalone components make dependencies explicit and simplify composition and lazy loading.

5. Explain lifecycle hooks.

Common hooks include ngOnChanges, ngOnInit, ngAfterViewInit and ngOnDestroy. Constructor is for construction/DI; ngOnInit is initialization after inputs are initialized; ngOnDestroy is cleanup.

6. constructor vs ngOnInit?

Constructor runs during class creation and is commonly used for DI. ngOnInit is an Angular lifecycle hook and is appropriate for initialization that depends on Angular-initialized inputs.

7. What are Angular binding types?

Interpolation and property binding move data to the view; event binding sends UI events to component logic; two-way binding combines property and event binding.

8. Property vs attribute binding?

Property binding changes the live DOM property, e.g. [disabled]. Attribute binding changes an HTML attribute, e.g. [attr.aria-label].

9. What are directives?

They add behavior to elements. Attribute directives modify behavior/appearance; structural/control-flow constructs determine what is rendered. Components are a specialized directive with a template.

10. What are pipes? Pure vs impure?

Pipes transform values for display. Pure pipes run when relevant input values/references change; impure pipes can run on every change-detection cycle and can be more expensive.

11. Explain Angular DI.

Angular creates/provides dependencies such as services and injects them into classes. This improves reuse, separation of concerns and testing.

12. What does providedIn:'root' mean?

The service is registered with the root injector and is normally available application-wide, commonly giving one root-level instance.

13. Explain routing.

Routes map URLs to components or lazy route configurations. RouterLink navigates declaratively, Router navigates programmatically, and ActivatedRoute exposes current route information.

14. ActivatedRoute snapshot vs paramMap?

snapshot gives the current value once. paramMap is reactive and is useful when route parameters can change while the same component instance remains active.

15. Path vs query parameter?

Path parameters identify route/resource segments such as /users/10. Query parameters represent optional state such as filtering, sorting or pagination.

16. How do you lazy load in modern Angular?

Use `loadComponent` for a standalone component or `loadChildren` for a lazy route configuration. This splits code and reduces initial download.

17. How is HttpClient configured in standalone Angular?

Use `provideHttpClient(...)` in application providers. Features such as functional interceptors can be added with `withInterceptors(...)`.

18. What is an HTTP interceptor?

It can inspect/modify outgoing requests and responses. Common uses are JWT headers, logging, common error handling and correlation IDs.

19. How do you attach JWT to every request?

Use an interceptor, clone the immutable `HttpRequest`, add `Authorization: Bearer` , and pass the cloned request to `next()`. Handle expiry according to the auth design.

20. What is a Signal in Angular 20?

A reactive wrapper around a value. Read it by calling it, e.g. `count()`. Angular tracks where it is read and can update affected consumers when it changes.

21. Signal vs normal variable?

Signals explicitly participate in Angular's reactive dependency tracking; ordinary variables do not provide the same signal-based tracking.

22. computed vs effect?

`computed` creates derived reactive state. `effect` performs side effects when signals it reads change. Use `computed` for derived values and `effect` for synchronization/side effects.

23. What is linkedSignal?

It represents state related to another reactive value but still independently writable. It is useful when a value normally follows a source but may be adjusted/reset.

24. Are Signals a replacement for RxJS?

No. Signals are excellent for synchronous reactive UI state and derived state; RxJS is designed for asynchronous/event streams and operators such as `switchMap` and `combineLatest`. They complement each other.

25. When use switchMap?

For latest-request-wins behavior, such as search/typeahead. A new source value unsubscribes from the previous inner observable.

26. switchMap vs mergeMap vs concatMap vs exhaustMap?

`switchMap` keeps the latest; `mergeMap` allows concurrent inner work; `concatMap` queues sequentially; `exhaustMap` ignores new emissions while current work is active.

27. What is forkJoin?

It waits for all supplied observables to complete and emits their final values together. It is useful for independent HTTP calls that all must finish.

28. Subject vs BehaviorSubject?

`Subject` has no stored current value for new subscribers. `BehaviorSubject` stores the latest value and emits it immediately to a new subscriber.

29. Explain change detection.

Angular checks whether template-bound state needs DOM updates. `OnPush` narrows when components need checking; signals provide dependency tracking for signal state read by templates.

30. Default vs OnPush?

Default checks more broadly during change-detection cycles. OnPush allows Angular to skip subtrees until relevant inputs/events/signals or explicit marking require checking.

31. Signals with OnPush?

When an OnPush template reads a signal, Angular tracks that dependency. Changing the signal marks the relevant component for update.

32. What is zoneless Angular?

It removes the requirement for ZoneJS to discover asynchronous activity as the main trigger. Angular can use explicit framework notifications such as signal updates. Current Angular documentation lists zoneless as production-ready.

33. Explain @if, @for, @switch.

Modern built-in control flow for conditional rendering, iteration and multiple cases. @for also supports tracking items with a stable key.

34. Why track in @for?

A stable key lets Angular associate data items with existing DOM nodes, reducing unnecessary DOM creation when collections change.

35. What is @defer?

It lets Angular defer loading/rendering non-critical template sections until triggers such as viewport, interaction or idle time.

36. How optimize a large Angular table?

Use server-side pagination/filtering/sorting, OnPush, stable tracking keys, virtual scrolling or a suitable grid, debounced search, lazy loading and a controlled DOM size.

37. How handle API pagination?

Send page/size or cursor parameters to the backend and render only the required data. Keep pagination in route query parameters when it should be shareable.

38. What are reactive forms?

Forms whose state and validation are explicitly modeled in TypeScript with FormControl, FormGroup and FormArray. They are strong for complex/dynamic forms.

39. Template-driven vs reactive forms?

Template-driven is simpler and template-centric. Reactive forms are explicit, scalable and easier to test for complex/dynamic forms.

40. What is ControlValueAccessor?

The bridge between Angular forms and a custom control. Implement writeValue, registerOnChange, registerOnTouched and optionally setDisabledState.

41. What is ViewChild?

It obtains a reference to a child component/directive/view element. Use it carefully and prefer normal data binding when a direct reference is unnecessary.

42. @Input/@Output vs input()/output() in Angular 20?

Decorators remain valid. Modern Angular provides signal-based input() and output() APIs. input() returns an InputSignal and supports input.required().

43. Parent-child communication?

Parent to child through inputs; child to parent through output events. Unrelated components can share state through a service, signals, RxJS or state management.

44. Component vs service responsibility?

Components coordinate UI and interaction. Services should contain reusable business/data-access logic such as HTTP calls.

45. Do you need NgRx for every app?

No. Local state can use signals; shared state can use services/signals/RxJS. NgRx is useful when application state and cross-feature coordination become sufficiently complex.

46. How prevent Angular memory leaks?

Clean up subscriptions/timers/listeners. Prefer async pipe or takeUntilDestroyed/DestroyRef where appropriate, and avoid retaining component references after destruction.

47. What is AOT?

Ahead-of-Time compilation compiles Angular templates during the build, improving production startup and enabling build-time optimization.

48. How debug a slow API/page?

Measure first: Network timings, payload size, backend latency, duplicate requests, rendering and change-detection cost. Then fix the measured bottleneck.

49. Explain Angular to Spring Boot flow.

Component/event or reactive stream -> service -> HttpClient -> interceptor -> HTTP -> Spring
Security/controller/service/repository/DB -> JSON response -> HttpClient -> signal/component state -> template.

50. How explain your Angular 20 project?

Pick one real feature and explain route -> component -> signals/inputs -> service -> HttpClient/interceptor -> Spring Boot API -> response -> state update -> UI. Mention RxJS usage, forms, error handling and performance decisions.

2-HOUR CRASH REVISION

- 20 min: JavaScript — var/let/const, closures, promises/async-await, event loop, map/filter/find/reduce, spread/destructuring, call/apply/bind.
- 30 min: Angular core — components, standalone, lifecycle, DI, services, routing, guards, forms, HttpClient, interceptors, parent-child communication.
- 35 min: Angular 20 — signals, computed, effect, linkedSignal, input(), output(), OnPush, @if/@for/@switch, @defer, zoneless.
- 20 min: RxJS — Observable, Subject/BehaviorSubject, map, switchMap, mergeMap, concatMap, exhaustMap, forkJoin.
- 15 min: Your project — explain one feature end-to-end and one performance/debugging problem.

Highest priority: Q20–35, Q42, Q48–50, plus JavaScript event loop and RxJS operator differences.